HAFTA 4 · KRİPTOLOJİ

Uygulamalı kriptografi

Veriyi şifrele, çöz, doğrula.



NEDEN ŞİFRELEMEK İSTERİZ?

Veriyi alan herkes okuyamasın

GÖNDERİRKEN

Özel bir mesaj

Alice'in Bob'a gönderdiği notu ağdaki başka biri okuyamasın.

SAKLARKEN

Yedek dosyası

Yedeğin kopyasını alan kişi içindeki belgeleri açamasın.

Açık veri

```
Bulusma: 14.30
```

→

Key ile şifrele

```
Okunamayan içerik
```

→

Doğru key ile çöz

```
Bulusma: 14.30
```

TARİHÇE · SEZAR

Bir mesajı üç harf kaydıralım

Açık mesaj

MERHABA

→

Key

+3

→

Şifreli mesaj

PHUKDED

M → P E → H R → U

Geri okumak için her harfi 3 adım geri al.

Enigma: harflerden makinelere

II. Dünya Savaşı

Tuşa basılır.

Rotorlar döner.

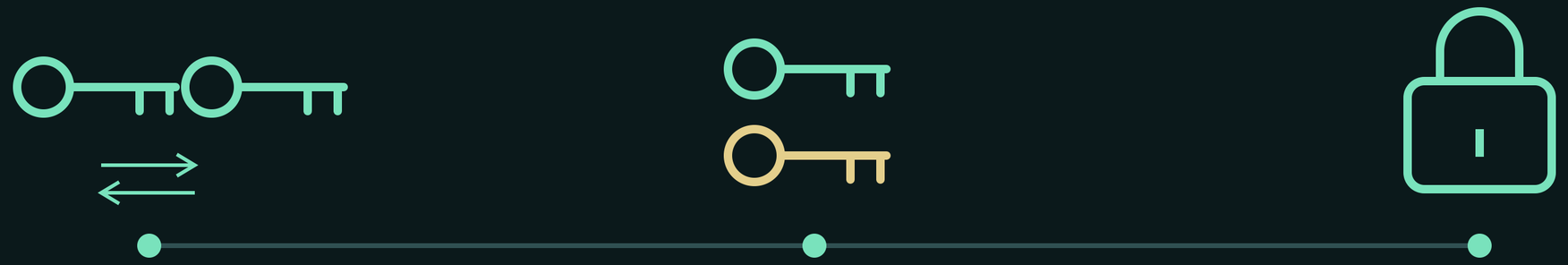
Başka bir harf yanar.

Anahtar: makinenin ayarları.



Fotoğraf: Punishar · kırpma: Hohum · CC BY-SA 4.0

Modern kriptografiden üç dönüm noktası



1976

Diffie–Hellman

Shared secret oluşturma

1978

RSA makalesi

Public / private key

2001

AES

Simetrik şifreleme standardı

KRİPTOLOJİ

Aynı not, iki farklı soru

KRİPTOGRAFİ

Notu nasıl korurum?

Buluşma saatini key ile şifrele.

KRİPTANALİZ

Nasıl okuyabilirim?

Zayıf key'i tahmin etmeyi dene.

Kriptoloji, bu iki bakışı kapsar.

Hangi araç, hangi iş için?

KRİPTOGRAFİ ARAÇLARI

SİMETRİK ŞİFRELEME

Aynı secret key

Veriyi şifrele ve geri çöz

Not · dosya · yedek

TLS'te uygulama verisi

AES-GCM

Alice: K ile şifrele

Bob: aynı K ile çöz

ASİMETRİK YÖNTEMLER

Public / private key çifti

Şifreleme · RSA-OAEP

Bob'a özel bir not gönder.

Dijital imza · Ed25519

Güncellemenin yayıncısını doğrula.

Key exchange · X25519

Alice ve Bob ortak secret oluştursun.

HASHING

Tek yönlü özet

Dosya özeti · SHA-256

Dosya değişti mi?

Parola · Argon2id + salt

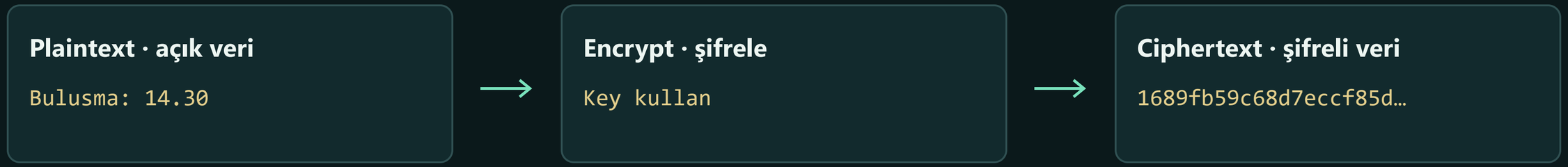
Girilen parola doğru mu?

Hash, şifreleme değildir.
Özetten veri geri çözülmez.

TLS örneği: kimlik doğrulama + key exchange → simetrik veri şifreleme

ENCRYPTION / DECRYPTION

Buluşma saatini gizleyelim



Decrypt: doğru key ile “Bulusma: 14.30” mesajını geri al.

Alice ve Bob aynı key'i kullanır

Alice

K ile şifrele

→

Gönderilen

Şifreli not

→

Bob

Aynı K ile çöz

Somut örnek

Bulusma: 14.30 → şifreli not → Bulusma: 14.30

Örnek araç: AES-GCM

SECRET

Hangi değeri paylaşmamalıyım?

SECRET

AES key

Key'i alan kişi şifreli notu açabilir.

PAYLAŞILABİLİR

Şifreli not

Tek başına okunabilir mesaj değildir.

Secret: gizli tutulması gereken değer. API token'ı da bir örnektir.

SİMETRİK · KEY DAĞITIMI

Key'i de mesaja eklesek?

Şifreli not + onu açan key.
İkisini alan kişi notu okur.



Anahtar paspasın altında.

ASİMETRİK ŞİFRELEME

Bob'un iki farklı key'i var

PUBLIC KEY

Herkes alabilir

Alice bu key ile Bob'a not şifreler.

PRIVATE KEY

Bob'da kalır

Şifreli notu Bob bu key ile açar.

Örnek araç: RSA-OAEP

ASİMETRİK · ÇALIŞAN ÖRNEK

Aynı buluşma notunu RSA ile gönderelim

Alice

`Bob'un public key'i`
`Encrypt`

→

Gönderilen

`Şifreli not`

→

Bob

`Kendi private key'i`
`Decrypt`

Yerel RSA-OAEP demosu

```
Çözülen not: Bulusma: 14.30
```

Tam çalışan kod: demo.py → rsa_example

Secret sayıları göndermeden anlaşalım

Herkesin bildiği iki sayı: 5 ve 23

ALICE

Secret sayısı: 6

5^6'nın 23'e bölümünden kalan: 8
Bob'a 8 gönderir.

BOB

Secret sayısı: 15

5^{15}'in 23'e bölümünden kalan: 19
Alice'e 19 gönderir.

Küçük sayılar yalnız anlatım için; gerçek güvenlik sağlamaz.

İkisi de aynı secret'a ulaştı: 2

ALICE HESAPLAR

$19^{6} \bmod 23 = 2$

Bob'dan gelen 19 + kendi secret sayısı 6

BOB HESAPLAR

$8^{15} \bmod 23 = 2$

Alice'ten gelen 8 + kendi secret sayısı 15

Ortak sonuç

`2 ağdan gönderilmedi. İki tarafta ayrı hesaplandı.`

Gerçekte: uygun büyük parametreler + key türetme + kimlik doğrulama.

GÜNLÜK ÖRNEK · HTTPS

Tarayıcı bu işleri birlikte kullanır

Kiminle konuşuyorum?

`Sertifikayı doğrula`

→

Ortak key oluştur

`Key exchange`

→

Veriyi taşı

`Simetrik şifreleme`

HTTPS = HTTP + TLS. Örnek: giriş formundaki parolanın sunucuya gitmesi.

TLS 1.3 örneği: sertifika + ECDHE (X25519 gibi) + AES-GCM.

RNG · RANDOM NUMBER GENERATOR

Yeni bir değer üretmem gerekiyor

OYUN

Zar sonucu

1 ile 6 arasında bir sayı

UYGULAMA

Parola sıfırlama token'ı

Başkasının tahmin edemeyeceği değer

İkisi de sayı / bayt üretir. Güvenlik ihtiyacı aynı değildir.

Aynı seed, aynı sayı dizisi

import random · seed (tohum) = 42

```
1  rng = random.Random(42)
2  [rng.randrange(100) for _ in range(5)]
```

İki ayrı üreteci 42 ile başlatınca

```
Alice: [81, 14, 3, 94, 35]
Bob: [81, 14, 3, 94, 35]
```

Testte yararlı: aynı deneyi tekrar edebilirim.

PRNG · RESET ÖRNEĞİ

Bob'un reset kodunu tahmin ettim

Hatalı uygulama

```
Seed = 42
Üretim sırası biliniyor
```

→

Aynı diziyi üret

```
Bob'un kodu:
216739
```

→

Sonuç

```
Bob'un parolası
başkası tarafından değişebilir
```

Kod rastgele görünüyor. Ama aynı başlangıçla tekrar üretilebiliyor.

CSPRNG · GÜVENLİ ÜRETİM

Reset token'ını secrets ile üret

Python · parola sıfırlama bağlantısı için örnek

```python
import secrets
token = secrets.token_urlsafe(32)
```

Neden? Token'ı tahmin eden kişi Bob'un hesabını alabilir.

secrets, rastgele baytları işletim sisteminden ister.
32 bayt → 256 bit: 6 rakamdan çok daha geniş aday alanı.

Token: kullanıcıya bağlı, kısa ömürlü ve tek kullanımlık olmalı.

İşletim sistemi rastgeleliği nereden bulur?

Entropi: saldırganın önceden bilemediği belirsizlik.

1 · Kaynaklar

Donanım gürültüsü
Olayların geliş zamanları

→

2 · İşletim sistemi

Girdileri birleştirir
CSPRNG'yi besler

→

3 · Güvenli üreteç

Secret iç durumdan
rastgele baytlar üretir

Çıktıları bilmek, sıradaki çıktıyı hesaplamaya yetmemeli.

Örnek: donanım olayları arasındaki çok küçük zaman farkları.
Kullanılan kaynaklar işletim sistemine ve donanıma göre değişir.

Basitleştirilmiş akış · Python · işletim sisteminden rastgelelik · Linux · rastgelelik kaynağının işleyişi

CSPRNG · BAYTTAN TOKEN'A

secrets burada ne yapıyor?

1 · Python ister

`token_urlsafe(32)`
`32 rastgele bayt ister`

→

2 · Sistem üretir

`CSPRNG'den`
`32 bayt döndürür`

→

3 · Python biçimlendirir

`Base64url ile`
`linke uygun yazıya çevirir`

32 bayt = 256 bit. Base64url sadece yazım biçimidir.

Alice kendi reset token'ını biliyor

`Bu bilgi, Bob'un token'ını hesaplamaya yetmemeli.`

Güvenlik beklentisi: çok seçenek + tahmin edilemez üretim.

Python · secrets · Python · işletim sisteminden rastgelelik

NONCE · NUMBER USED ONCE

Her şifreli nota yeni bir değer

	Key	Nonce
Not 1: Bulusma: 14.30	K	N1
Not 2: Bulusma: 15.00	Aynı K	Farklı N2

AES-GCM kuralı: aynı key ile nonce tekrar etmemeli.

Key secret'tır. Nonce, şifreli notla birlikte saklanabilir.

Nonce'u nasıl üreteceğiz?

Her yeni AES-GCM şifrelemesinden önce

```
1 nonce = secrets.token_bytes(12)
```

SECRETS

Nasıl üretiyorum?

Güvenli rastgelelik kaynağıyla

NONCE

Nerede kullanıyorum?

Bu şifreleme işleminin girdisi olarak

Demo 12 bayt kullanır. Rastgele üretimde çakışma ihtimali sıfır değildir.

NONCE · TEKRAR

Her seferinde aynı nonce?

Key aynı.
Nonce aynı.
Mesaj farklı.
AES-GCM için bu kullanım hatalı.



“Testte açıldı ama…”

AES-GCM · ENCRYPT

Buluşma notunu gerçekten şifreleyelim

Python · lab/hafta04/demo.py

```python
import secrets
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
key = AESGCM.generate_key(bit_length=256)
nonce = secrets.token_bytes(12)
message = b"Bulusma: 14.30"
sealed = AESGCM(key).encrypt(nonce, message, None)
```

key: secret · nonce: her yeni şifrelemede yeni değer

AES-GCM · DECRYPT

Doğru key ile not geri geliyor

Python · lab/hafta04/demo.py

```python
opened = AESGCM(key).decrypt(nonce, sealed, None)
print(opened.decode())  # Bulusma: 14.30
```

Çalışan örneğin çıktısı

```
Bulusma: 14.30
```

DEĞİŞTİR VE DENE

Yanlış key ile ne olur?

Denediğimiz	Sonuç
Doğru key + özgün paket	Not açılır
Başka bir key	InvalidTag → ret
Şifreli pakette bir bayt değişikliği	InvalidTag → ret

Hatalı paketten çıkan veriyi kullanmaya devam etmeyiz.

ENCODING · BASE64

Bunu açmak için key gerekmiyor

Python · lab/hafta04/demo.py

```python
import base64
message = b"amount=100&to=alice"
encoded = base64.b64encode(message)
decoded = base64.b64decode(encoded)
```

Gösterim değişti; secret eklenmedi

```
YW1vdW50PTEwMCZ0bz1hbGljZQ==
→ amount=100&to=alice
```

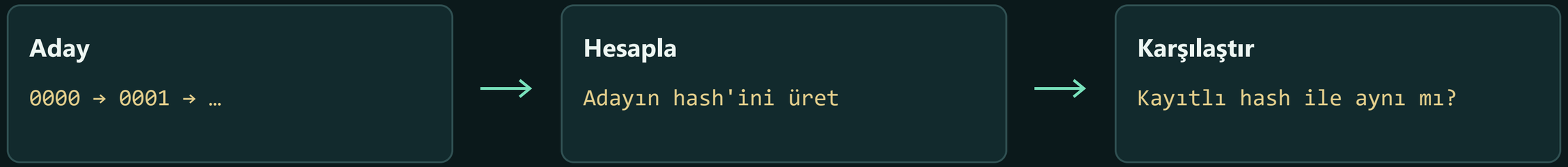
100 yerine 900 yazarsam özet değişir

Mesaj	SHA-256 · ilk 16 hex karakter
amount=100&to=alice	5dcb6d1ad0b99452…
amount=900&to=alice	17e9cfefcfb4faeb…

Örnek: dosyayı güvenilir bir kaynaktan aldığım özetle karşılaştırırım.

BRUTE FORCE · TÜM ADAYLARI DENE

Hash'i geri açmak yerine aday deneriz



4 rakam: 10^4 = 10.000 aday. 8 rakam: 10^8 = 100 milyon aday.

Offline: ele geçmiş hash üzerinde yerel deneme. Online: giriş sunucusuna istek; deneme sınırları devrede.

ADAY ALANI

256 bit çıktı, kaç farklı aday?

PIN yalnız 4 rakamsa:
SHA-256(PIN) için hâlâ
10.000 seçenek var.



Uzun hash ≠ çok seçenek.

2026 · YAYIMLANAN BENCHMARK

Bir ekran kartı, farklı hash hızları

Hash türü	1 × RTX 5090 · deneme / saniye
MD5 (raw)	219,5 milyar
NTLM	340,1 milyar
SHA-256 (raw)	27,6816 milyar

Hızlı genel hash'ler, parolayı tahmin etmeyi de hızlandırır.

Specops · 8 Nisan 2026 · Hashcat karşılaştırması · Hashcat · H/s = saniyedeki hash denemesi

2026 · ÖLÇÜM RAPORU

bcrypt için maliyet ayarını da söylemeliyiz

Raporlanan toplam hız

138.675 deneme / saniye

DONANIM

16 × RTX 5090

İki ayrı 8 GPU'lu sunucu

PAROLA HASH'İ

bcrypt · cost 10

Maliyet ayarı her tahmini yavaşlatır.

Hive Systems · 14 Temmuz 2026 · Teknik rapor · HS-PWD-2026-001 · Offline deneme

Kaç aday var, saniyede kaçını deniyoruz?

`Tam tarama süresi ≈ aday sayısı ÷ 138.675`

Rastgele üretilen parola	Aday sayısı	Tam tarama
8 rakam · 0–9	10^8	≈ 12,0 dakika
8 küçük harf · a–z	26^8	≈ 17,4 gün
10 küçük harf · a–z	26^{10}	≈ 32,3 yıl

bcrypt cost 10 · 16 × RTX 5090 · Sabit hız varsayımı. Rastgele hedefte ortalama süre yaklaşık yarısıdır.

Hesabın bir kısmını önceden yapsak?

Zinciri üret

`parola → hash → aday → …`

→

Tabloyu sakla

`Zincirin başı ve sonu`

→

Hash ele geçince

`Eşleşen zinciri yeniden üret`

Örnek: aynı salt'sız hash yöntemiyle saklanan parolalarda önceden hazırlanmış tablo tekrar kullanılabilir.

Rainbow table: hash + reduction zincirleriyle zaman / bellek takası. Her hash'i tek tek saklayan düz liste değildir.

SALT · HER KAYIT İÇİN FARKLI

Aynı parola, farklı kayıt

Kullanıcı	Parola	Salt	Saklanan sonuç
Alice	kahve2026	S1	ParolaHash(parola, S1)
Bob	kahve2026	Farklı S2	Farklı hash çıktısı

Salt secret değildir; hash ile saklanır.
Aynı hazır tabloyu tüm kullanıcılarda kullanmayı engeller.

Salt tek başına brute force'u durdurmaz. Argon2id gibi maliyetli parola hash'iyle birlikte kullanılır.

100 liralık mesajı 900 yapmayı deneyelim

Gönderen

```
amount=100
K ile tag üret
```

→

Değişiklik

```
amount=900
Eski tag duruyor
```

→

Alıcı

```
K ile kontrol
False → ret
```

HMAC içeriği gizlemez. Mesaj ve tag açıkça taşınabilir.

Çalışan örnek: demo.py → hmac

Uygulama güncellemesi kimden geldi?

Yayıncı

Private key ile imzala

→

İndirilen

Dosya + imza

→

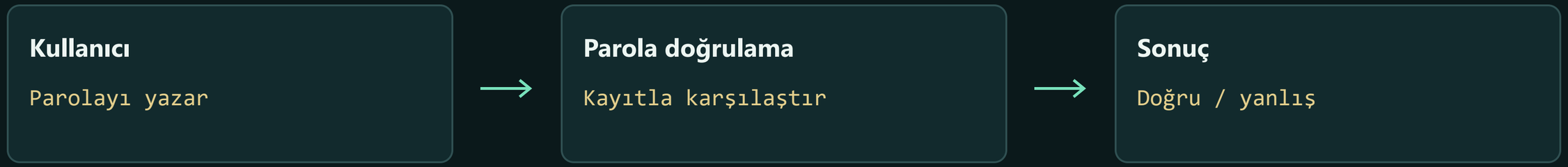
Kullanıcı

Public key ile doğrula

Ed25519 demosu

Özgün dosya: doğrulandı
Değişen dosya: InvalidSignature

Girişte parolayı geri okumamız gerekmiyor



Yeni parola kaydı için Argon2id gibi parola hashing araçları.

Buluşma notu: geri çözülecek veri. Parola: doğrulanacak giriş.

SECRET YÖNETİMİ

Key hangi ekranda görünmemeli?

HATALI

Hata sayfası / log / repo

Key görünürse şifreli not da açılabilir.

UYGUN YER

Yetkisi sınırlı secret deposu

Yalnız gereken uygulama erişir.

Key sızdıysa: erişimi kapat, key'i yenile, etkilenen veriyi değerlendir.

SORU · CEVAP

Key'i mi bulduk, algoritmayı mı kırdık?

Az seçenek: tahmin edilebilir.
Sızan key: not açılabilir.
Güvenlik, kullanım biçimine de bağlı.



İki kişi, bir klavye. Daha büyük key alanı sayılmaz.

2026 hızları ve kavramların kaynakları

Specops · 8 Nisan 2026 · Hashcat karşılaştırması

Hive Systems · 14 Temmuz 2026 · Teknik rapor

Oechslin · Rainbow tables · CRYPTO 2003

OWASP · Password Storage

OWASP · Parola sıfırlama token'ları

RFC 8446 · TLS 1.3

2026 hız kaynakları teknik benchmark yayınlarıdır; hakemli makale değildir. Süre tablosu N / hız hesabıdır.

BAŞVURU · KOD

Örneklerin belgeleri

cryptography · AESGCM

cryptography · X25519 anahtar anlaşması

Python · secrets

Python · random / PRNG

Python · hmac

cryptography · Ed25519

Dersin kaynakları

ASD · Enigma'nın tarihi

Diffie–Hellman · New Directions in Cryptography (1976)

Rivest–Shamir–Adleman · RSA makalesi

NIST FIPS 197 · AES (2001)

NIST SP 800-38D · GCM

OWASP · Password Storage

OWASP · Hata yanıtlarında bilgi sızıntısı